Title: Bird Brain (Team 8 Rec. 12)

Who:

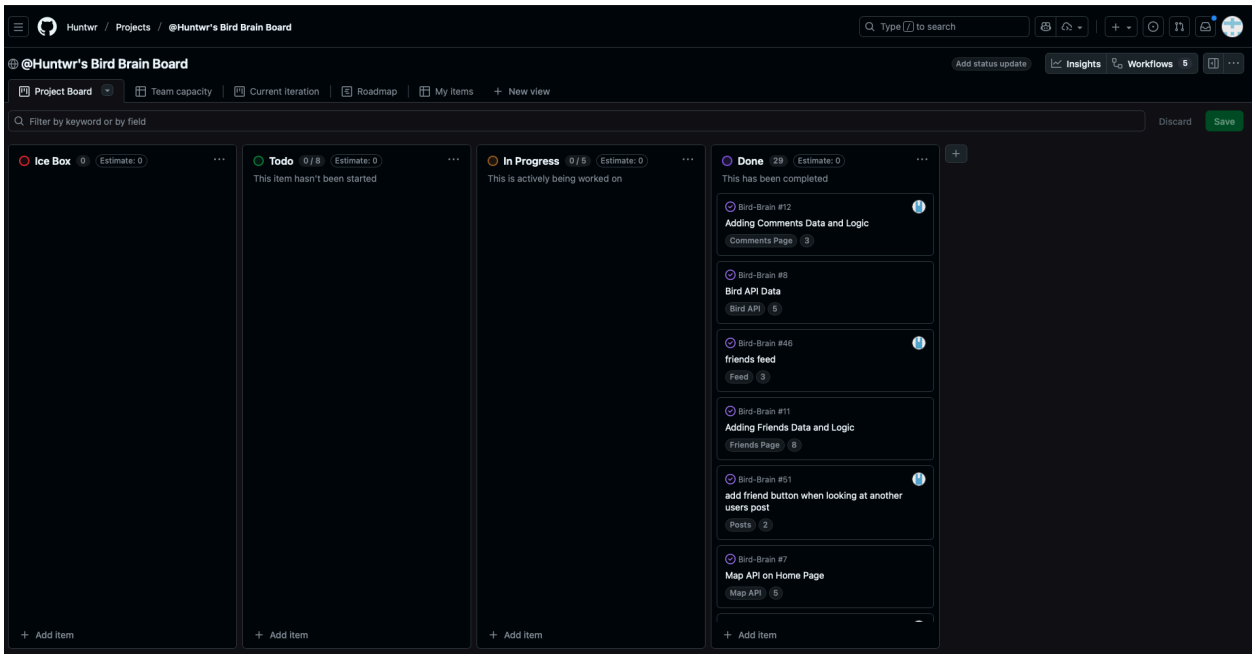
Name:	Email:	Github:
Aaron Duong	aadu7536@colorado.edu	Aaronduo7536
Koa Lister	pali8174@colorado.edu	koalister
Vinnie De Lisi	vide1065@colorado.edu	VinnieDeLisi
Jackson Gothie	jago6572@colorado.edu	jackg22
Hunter irish	huir2366@colorado.edu	Huntwr

Project Description: A 200 word summary of the project - Hunter

Our Project is a web application/social media platform for bird watchers to share and talk about their recent finds with their friends and the world. We wanted the app to be simple, familiar, and fun. The app is made for users to simply log in and start exploring the world of bird watching around them. The app allows users to record information about their most recent sightings such as location, species, date and time, and descriptions. Furthermore, the app is designed so that even if you don't know exactly what type of bird you spotted, it can help you by using the Gemini API to suggest some options of what species it might be. Included in this is the eBird API, which helps users by providing a long detailed list of different species options so that any user can find exactly the bird they are looking for. We included a map using the Mapbox API into the app where users can see what was most recently seen around them and potentially friend fellow bird watchers in their area. The app is built for professional bird watchers and amateurs alike to enjoy and share the world of bird watching.

Project Tracker - GitHub project board:

https://github.com/users/Huntwr/projects/1/views/1



Video:

https://drive.google.com/file/d/1rOAa922UfnyuVtoYGObPd4R_10GYPgpS/view?usp=sharing

VCS:

https://github.com/Huntwr/Bird-Brain

Contributions:

Hunter: I worked on a few things, I set up the initial file structure and getting the initial files working and running such as the docker setup, Node Js set up, and the handlebars setup. I also created the log in and sign up pages and added functionality with Node Js. I also did the home page and made the main feed and the friends feed. I also created the main color scheme for the project. Finally I did a lot of bug fixes all across the app to get everything working smoothly

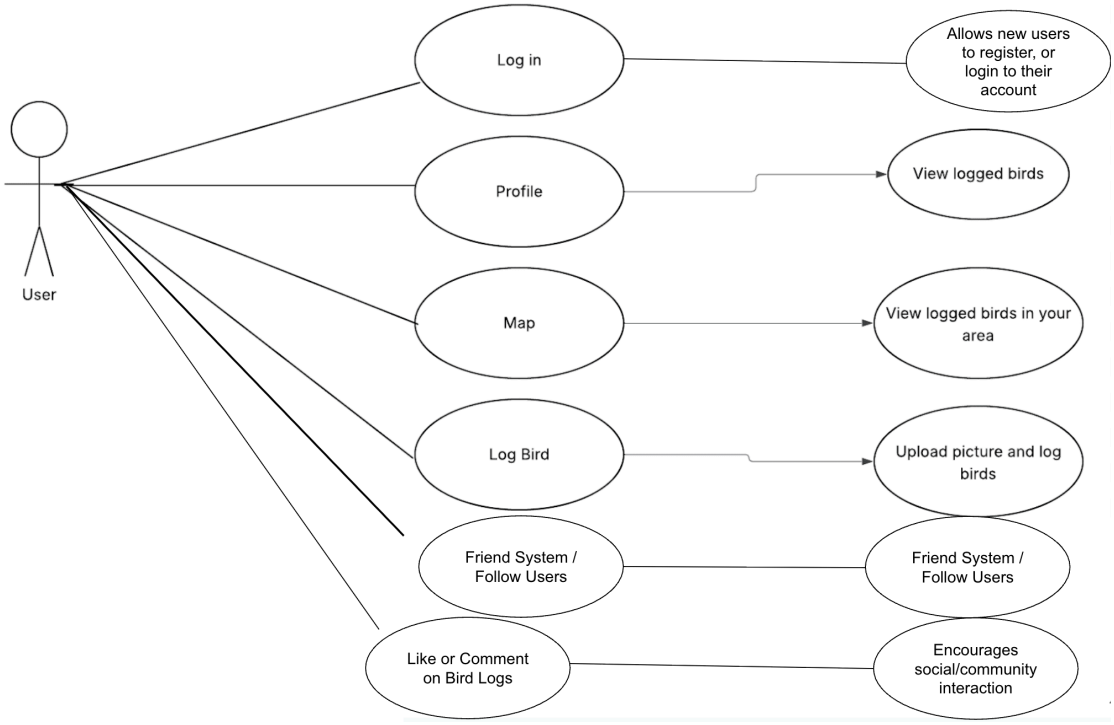
Koa: I worked totally on the friends page and the logic behind getting a friends system to work on our website. The friends page includes complete functionality that allows users to send and receive friend requests and see their friends profiles. Additionally, you are able to favorite friends within the friends page. I used a database in order to keep track of users, friendships, and favorites. I also added an invite system that copies the link to the website to the user's clipboard.

Vinnie: I was responsible for the entirety of the profile page. This included displaying a default profile picture for every created account, allowing users to upload their own profile picture, and allowing them to input a description about themselves. I also implemented the profile page showing when the account was created, and made it so that any posts a user created would be shown on their profile page too. I used a database in order to store any user inputted information.

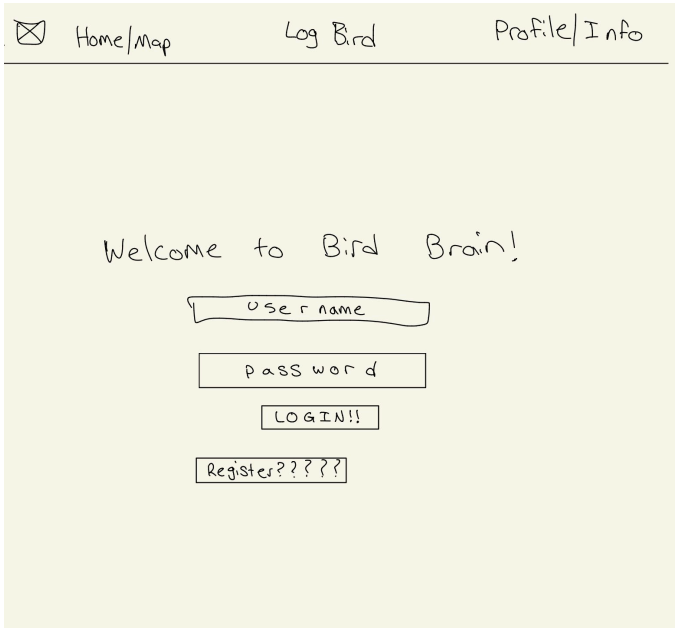
Jackson: I was responsible for the log-bird page. This page allows users, once logged in, to post a new bird to their account. This bird would then connect to the map API and display the bird's location on it. I was not responsible for the API but more handling the posts. I also handled the database where we stored user information and post information. Since another group member handled the login page, they handled the users table, while I constructed the "bird_sightings" table.

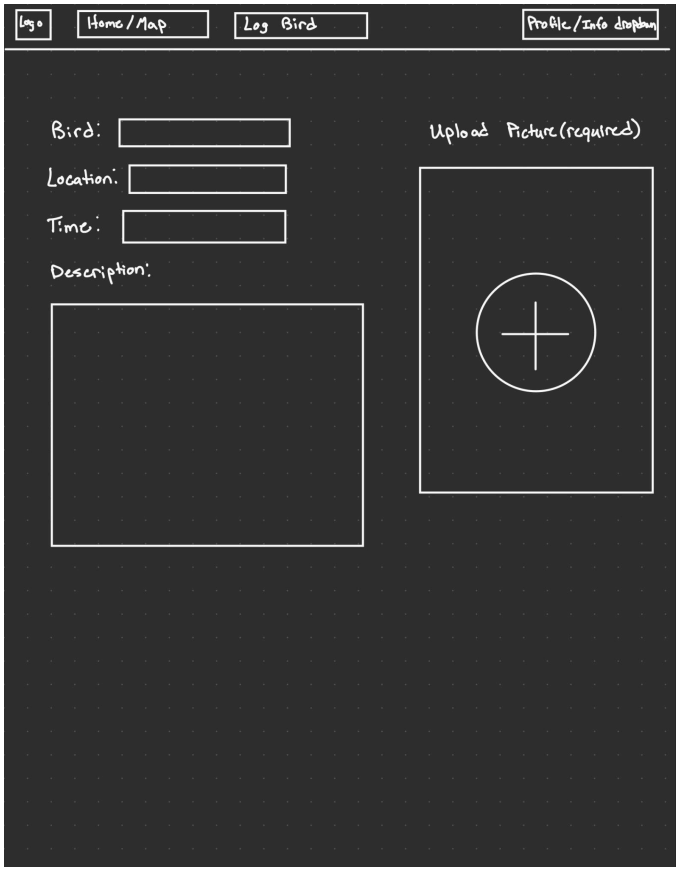
Aaron: I was focused on setting up our API tokens and actually implementing all API related features across the site. This included integrating the Mapbox API to display the main map, the eBird API for species data and options list, and the Gemini API for the quick bird survey. I also built the survey logic for both the bird survey system and the location survey logic. Lastly, I implemented the functionality for converting user entered general location into geographic coordinates and generating map pins for all bird postings.

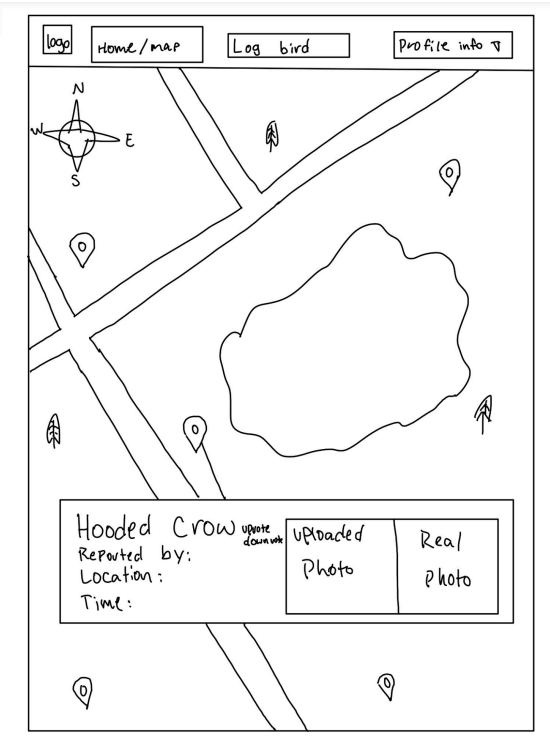
Use Case Diagram:

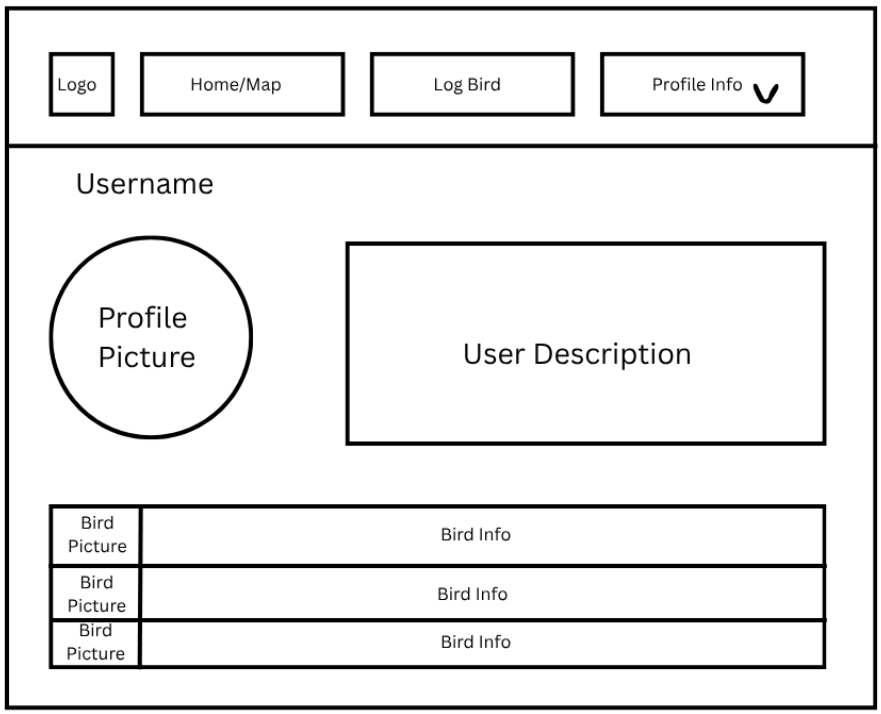


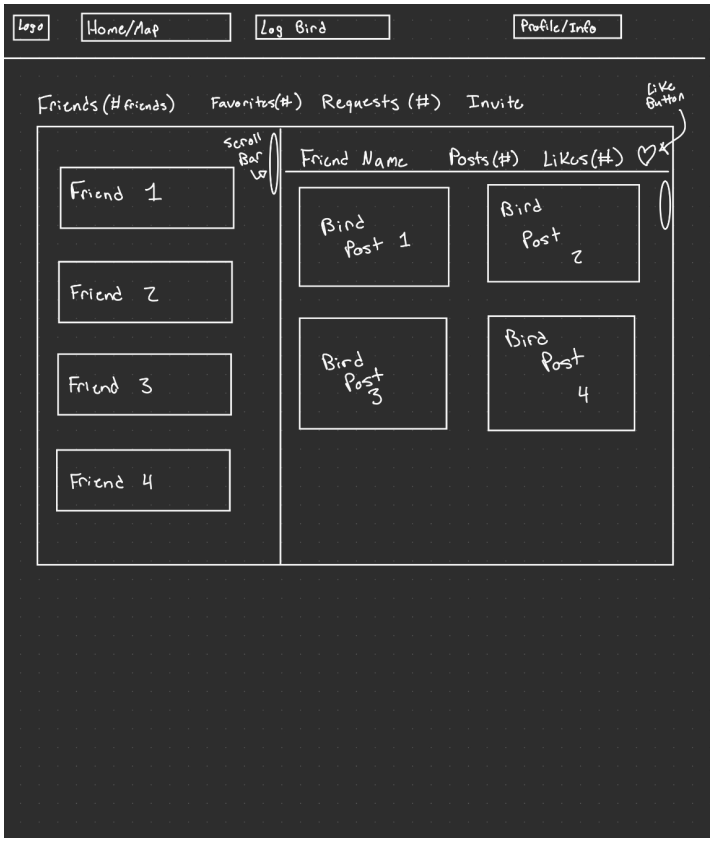
Wireframes:

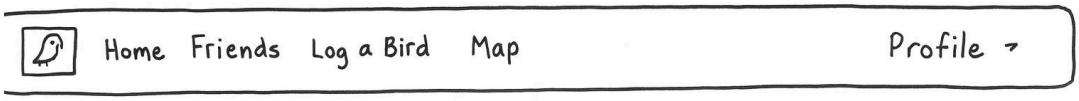

Home/Map
Log Bird
Profile/Info
Welcome to Bird Brain!
user name
password
LOGIN!!
Register?????

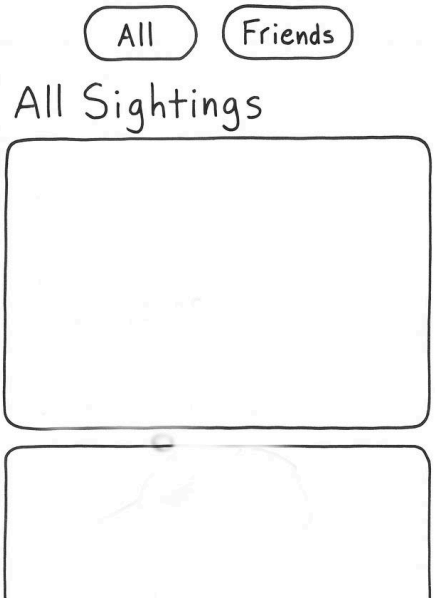

Logo
Home/Map
Log Bird
Profile/Info dropdown
Bird:
Location:
Time:
Description:
Upload Picture(required)

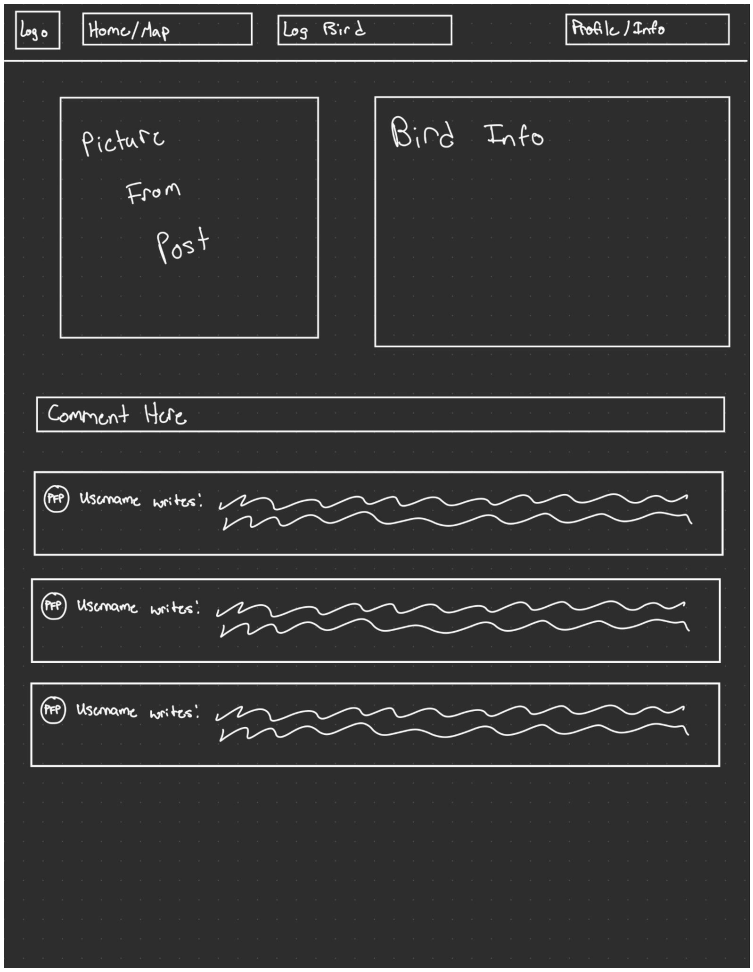

logo
Home / map
Log bird
Profile info
N
W
E
S
Hooded Crow
upvote downvote
Reported by:
Location:
Time:
Uploaded Photo
Real Photo


Logo
Home/Map
Log Bird
Profile Info
Username
Profile Picture
User Description
Bird Picture
Bird Info
Bird Picture
Bird Info
Bird Picture
Bird Info


Logo
Home/Map
Log Bird
Profile/Info
Friends (# friends)
Favorites(#)
Requests (#)
Invite
Like Button
Scroll Bar
Friend Name
Posts(#)
Likes(#)
Friend 1
Friend 2
Friend 3
Friend 4
Bird Post 1
Bird Post 2
Bird Post 3
Bird Post 4


Home
Friends
Log a Bird
Map
Profile


All
Friends
All Sightings


Logo
Home/Map
Log Bird
Profile/Info
Picture From Post
Bird Info
Comment Here
PFP
Username writes:
PFP
Username writes:
PFP
Username writes:

Test results:

Test Case 1 – User Registration and Login

Description: A new user should be able to create an account and then log in to access their profile.The system should store valid information, reject invalid input, and display clear feedback messages.

Test Environment: Localhost (Dockerized Node.js + PostgreSQL)

Test Data:

1. Valid: username = “birdlover”, password = “flyhigh123”

2. Invalid: blank fields or mismatched passwords

Test Steps:

Navigate to the registration page from the homepage.

Enter valid information and submit the form.

After registration, attempt to log in using the same credentials.

Attempt to submit the form with missing fields.

Expected Results:

- Successful registration redirects to the login page.

- Valid credentials on login open the user dashboard.

- Invalid or blank fields show an on-screen error message.

Observations test case 1

When the users are interacting with your application, you should be observing their actions. You should be taking notes of the following:

What are the users doing?

During the test, our test users were clicking around the landing page to find the registration link, filling out the sign up form, and then immediately trying to log in with the same credentials. One member also purposely left fields blank or entered mismatched passwords to see what happens.

What is the user's reasoning for their actions?

Most of them want to see how fast they can get from no account to being logged in with information showing. Only one user wanted to see fail cases, hence why he spent more time playing around with negative inputs.

Is their behavior consistent with the use case?

Yes, it matches the use case well. They follow the expected path of going from the home page to the registration page, signing up, and then logging in.

If there is a deviation from the expected actions, what is the reason for that?

The main deviations are small things. For example, some people would try to log in before registering because they assume they already have an account, or they double click buttons when the page feels a bit slow. That usually happens when the feedback is not instantly obvious or the button does not clearly show that something is in progress, so they repeat the action just to be sure.

Did you use that to make changes to your application? If so, what changes did you make?

Yes. Based on what we saw we made a few tweaks. We improved the error messages for blank fields and mismatched passwords so they are more specific and easy to understand. We also made sure the form keeps the entered username when there is an error, so users do not have to retype everything. Finally we adjusted the visual feedback on the register and login buttons so it is clearer that the request is being processed when they click.

Test Case 2 – Bird Posting and Viewing

Description: A logged in user should be able to create a bird sighting post that includes a title, short description, image, and location. The system should correctly store the post, display it in the feed, and persist it after refresh.

Test Environment:

Localhost (Dockerized Web App + PostgreSQL)

Test Data:t

Post title: “Blue Jay near Boulder Creek”

Description: “Saw a Blue Jay while walking near campus.”

Image: test_photo.jpg

Test Steps:

Log in as an existing user.

Navigate to the “Add Bird” page.

Enter all required fields using the test data.

Submit the form and return to the feed page.

Verify that the new post appears correctly.

Expected Results:

The new bird post appears immediately in the feed.

The uploaded photo, title, and description display correctly.

The post remains visible after refreshing the page.

Observations test case 2

When the users are interacting with your application, you should be observing their actions. You should be taking notes of the following:

What are the users doing?

For this test, users logged in and navigated to the add bird page, where they filled out the title and description right away, then uploaded the test image, and finally chose a location before submitting. After posting, they all went straight back to the feed to confirm their new sighting showed up. One user also refreshed the page, then clicked into different parts of the feed to double check that the post actually saved correctly.

What is the user's reasoning for their actions?

Their main goal was to see how easy it was to create a post and whether the app would display it right away. They wanted to confirm the image upload worked, since that part feels like the most important aspect to a user.

Is their behavior consistent with the use case?

Yes. Everything they did matches exactly what the use case describes. They followed the normal flow of logging in, creating a post, and checking that it appears in the feed.

If there is a deviation from the expected actions, what is the reason for that?

There was only one small deviation where one person tried to upload the image before filling out any text fields, just to see if the order mattered. This deviation mostly happened because they weren't sure how strict the form validation was, or because the UI didn't immediately show that the submission was processing.

Did you use that to make changes to your application? If so, what changes did you make?

Yes, we made similar changes to what was seen in test case 1. We added clearer form validation so the user knows exactly which fields need to be filled out before submitting. We also improved the loading feedback on the submit button so people don't double click.

Test Case 3 – Interactive Map of Sightings

Description: Users should be able to view an interactive map containing pins for all recorded bird sightings. Selecting a pin should reveal the sighting details, including the bird information and the user who posted it.

Test Environment:

Localhost (Dockerized Web App + Mapbox API)

Test Data:

Sample sighting locations with associated bird names

A newly added post with a unique test location

Test Steps:

Log in and navigate to the interactive map page.

Verify that existing bird sightings appear as map pins.

Click any pin to view its bird details and posting user.

Create a new sighting with a unique location.

Refresh the map and confirm the new pin appears.

Expected Results:

The map loads successfully with all existing pins visible.

Each pin displays the correct bird name, description, and user information.

Newly created sightings appear on the map after posting.

Observations test case 3

When the users are interacting with your application, you should be observing their actions. You should be taking notes of the following:

What are the users doing?

In this Test case the user went to the map page and looked around the map, clicking on pins and seeing the posts that other people have posted, they then clicked on one of the pins and clicked comment, they look at the comment page interacting with the post and even writing their own comment, they went back to the page and zoomed out to look at the globe and see if there were pins any where else in the world

What is the user's reasoning for their actions?

Their main goal was to see what birds were seen in their area, they wanted to see what types of birds are in their area and what birds their friends have seen recently in their area.

Is their behavior consistent with the use case?

Yes they did exactly as expected, they used the map exactly as intended≥

If there is a deviation from the expected actions, what is the reason for that?

There were actually no deviations in this case.

Did you use that to make changes to your application? If so, what changes did you make?

N/A

Test Case 4 – User Profile and Friends List Management

Description: Users should be able to view their profile information, see a list of friends, and send or accept friend requests. The system should correctly update the friend list, prevent duplicate requests, and display accurate status indicators.

Test Environment:

Localhost (Dockerized Web App + PostgreSQL)

Test Data:

Existing user accounts:

User A: “birdfan01”

User B: “campusbirder”

Friend request scenario: User A sends request → User B accepts.

Test Steps:

Log in as User A and navigate to the profile page.

Search for User B and send a friend request.

Log out and log in as User B.

Open the “Friends” or “Requests” page and accept the pending request.

Return to User A’s profile and verify the updated friend list.

Expected Results:

User A can successfully send a friend request to User B.

User B sees the request and can accept it.

Both profiles show each other under “Friends.”

Duplicate requests cannot be sent once the users are friends.

Observations test case 4

When the users are interacting with your application, you should be observing their actions. You should be taking notes of the following:

What are the users doing?

The users are adding each other as friends through the built in friend system, they are navigating to the friends tab pressing add friend, then typing in there friends email address and sending a request to their friend, than the second user is going to the friends tab and within that going to the requests tab and clicking accept to the friend request that is coming from their friend.

What is the user's reasoning for their actions?

They do this so they can see each other's posts easier and see them on the map. This is so they can see where they each saw birds recently.

Is their behavior consistent with the use case?

Yes the users did this exactly as expected and clicked exactly where they needed to, to send a friend request.

If there is a deviation from the expected actions, what is the reason for that?

There were only small deviations, for example the user tried to send multiple friend requests to the same person and this caused a server error.

Did you use that to make changes to your application? If so, what changes did you make?

We saw this and added a bug fix so that once you send a friend request if you try to send another to the same person you get a pop up telling you that you already sent a friend request.

Deployment: Link to deployment environment or a written description of how the app was deployed and how one might access/run the app. The app must be live, working, and accessible to your TA.

https://bird-brain-1.onrender.com